

The Computer Revolution

Progress in computer technology

- Underpinned by Moore's Law
- **Makes novel applications feasible**
- Computers in Automobiles
- Cell Phones
- Human Genome project
- World Wide Web
- Search Engines
- **Computers are pervasive**

Explanation:

Computers in Automobiles

reduce pollution, improve fuel efficiency via engine controls, and increase safety through blind spot warnings, lane departure warnings, moving object detection, and air bag inflation to protect occupants in a crash

Cell Phones

More than half of the planet having mobile phones, allowing person-to-person communication to almost anyone anywhere in the world

Human Genome Project

a global effort to identify the estimated 30,000 genes in human DNA to figure out the sequences of the chemical bases that make up human DNA to address ethical, legal, and social issues

The cost of computer equipment to map and analyze human DNA sequences was hundreds of millions of dollars. Since, costs continue to drop, we will soon be able to acquire our own genome, allowing medical care to be tailored to us

World Wide Web

has transformed our society. Web has replaced libraries and newspapers

Search Engines

As the content of the web grew in size and in value, many people rely on search engines

for such a large part of their lives that it would be a hardship to go without them

Classes of Computers

- **Personal computers**
- **Server computers**
- **Supercomputers, and**
- **Embedded computers**

Explanation:

Personal computers

- Computers designed for use by an individual
- General purpose, variety of software
- Subject to cost/performance tradeoff

Server computers

- Computers used for running larger programs for multiple users, often simultaneously
- Network based
- High capacity, performance, reliability
- Range from small servers to building sized

Supercomputers

- Consist thousands of processors and many terabytes of memory
- High-level scientific and engineering calculation
- Highest capability and cost but represent a small fraction of the overall computer market

Embedded computers

- Hidden as components of systems
- Largest class of computers and span the widest range of applications
- Strict power/performance/cost limitations

The PostPC Era

Cloud computing

- Term **Cloud** essentially used for the Internet
- Portion of software run on a PMD and portion run in the Cloud

Warehouse Scale Computers (WSCs)

Big datacenters containing 100,000 servers
Amazon and Google cloud vendors

Software as a Service (SaaS)

Delivers software and data as a service over the Internet
Web search and social networking

Understanding Performance

Algorithm

determines number of operations executed

Programming language, compiler, architecture

determine number of machine instructions executed per operation

Processor and memory system

determine how fast instructions are executed

I/O system (including OS)

determine how fast I/O operations are executed

Eight Great Design Ideas

1/ Design for *Moore's Law*

- Computer designs can take years, resources available per chip can easily double or quadruple between start and finish of project
- Anticipate where technology will be when design finishes

2/ Use *Abstraction* to simplify design

- Hide lower-level details to offer a simpler model at higher levels

3/ Make the *Common Case Fast*

- To enhance performance better than optimizing the rare case

4/ Performance *via Parallelism*

- A form of computation in which many calculations are carried out simultaneously

5/ Performance *via Pipelining*

- A particular pattern of parallelism
- A set of data processing elements connected in series, so that the output of one element is the input of the next one

6/ Performance *via Prediction*

- It can be faster on average to guess and start working rather than wait

7/ Hierarchy of memories

- The closer to the top, the faster and more expensive per bit of memory
- The wider the base of the layer, the bigger the memory

8/ Dependability via redundancy

- Design systems dependable by including redundant components to help detect failures, and to take over when failure occurs

Below Your Program

Applications software

- Written in high-level language

Systems software

- Compiler: translates HLL code to machine code
- Operating System:
 - Handling input/output
 - Managing memory and storage
 - Scheduling tasks & sharing resources

Hardware

- Processor, memory, I/O controllers

Levels of Program Code

High-level language

- Level of abstraction closer to problem domain
- Provides for productivity and portability

Assembly language

- Textual representation of Instructions

Hardware representation

- Binary digits (bits)
- Encoded instructions and data

Inside the Processor (CPU)

- **Datapath:** performs operations on data
- **Control:** sequences data path, memory, ...
- Cache memory
 - Small fast SRAM memory for immediate access to data

Abstractions

- **Abstraction helps us deal with complexity**
 - Hides lower-level details
- **Instruction Set Architecture (ISA) or Computer Architecture**
 - The hardware/software interface
 - Includes instructions, registers, memory access, I/O, and so on
- **Operating system hides details of doing I/O, allocating memory from programmers**

Networks

1/ Backbone of computer systems

■ Communication

- Information is exchanged between computers at high speeds

■ Resource sharing

- Rather than each computer having its own I/O devices, computers on the network can share I/O devices

■ Nonlocal access

- By connecting computers over long distances, users need not be near the computer they are using

2/

■ Local area network (LAN): Ethernet

- A network designed to carry data within a geographically confined area, typically within a single building – 40 gigabits/s

■ Wide area network (WAN): the Internet

- A network extended over hundreds of kilometers that can span a continent

■ Wireless network: Wi-Fi, Bluetooth

- Transmission rates from 1 to 100 million bits per second

Integrated Circuit Cost

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$
$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$
$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area} / 2))^2}$$

Response Time and Throughput

■ Response time

- How long it takes to do a task

■ Throughput

- Total work done per unit time e.g., tasks/transactions/... per hour

■ How are response time and throughput affected

by

- Replacing the processor with a faster version?
- Adding more processors?

■ We'll focus on response time for now...

Relative Performance

■ Performance = 1/Execution Time

■ "X is n times faster than Y"

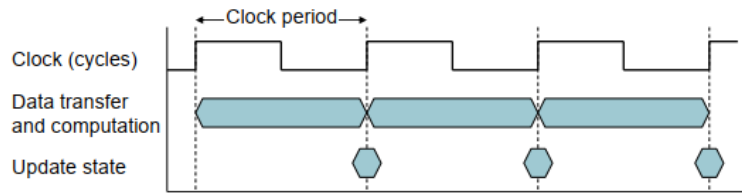
$$\frac{\text{Performance}_X}{\text{Performance}_Y}$$
$$= \frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

■ Example: time taken to run a program

- 10s on A, 15s on B
- $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15s / 10s = 1.5 = 1\frac{1}{2}$
- So A is $1\frac{1}{2}$ times faster than B

CPU Clocking

- Operation of digital hardware governed by a constant-rate clock



- Clock period: duration of a clock cycle
 - e.g., 250ps = 0.25ns = 250×10^{-12} s
- Clock frequency (rate): cycles per second (Hz)
 - e.g., 4.0GHz = 4000MHz = 4.0×10^9 Hz = 4.0×10^9 cps

CPU Time

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

- Performance can be improved by
 - Reducing number of clock cycles
 - Increasing clock rate
 - Hardware designer must often trade off clock rate against cycle count

CPU Time Example

- Computer A: 2GHz clock, 10s CPU time
- Designing Computer B
 - Aim for 6s CPU time
 - Can do faster clock, but causes $1.2 \times$ clock cycles
- How fast must Computer B clock be?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6\text{s}}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10\text{s} \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6\text{s}} = \frac{24 \times 10^9}{6\text{s}} = 4\text{GHz}$$

CPI Example

Computer A: Cycle Time = 250ps, CPI = 2.0

Computer B: Cycle Time = 500ps, CPI = 1.2

Same ISA

Which is faster? by how much?

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps} \quad \leftarrow \text{A is faster...}\end{aligned}$$

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}\end{aligned}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2 \quad \leftarrow \text{...by this much}$$

Instruction Count and CPI

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles Per Instruction}$$

$$\begin{aligned}\text{CPU Time} &= \text{Clock Cycles} \times \text{Clock Cycle Time} \\ &= \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time} \\ &= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}\end{aligned}$$

Instruction Count for a program

- Determined by program, ISA, and compiler

Average cycles per instruction

- Determined by CPU hardware
- If different instructions have different CPI

Average CPI gets affected by instruction mix
(dynamic frequency of instructions)

Is Moore's Law Ending?

